

QCC 测试报告自动化平台

项目说明文档

文档版本：v1.0

更新日期：2026年7月14日

文档目的：帮助汇报人全面掌握项目全貌，独立完成汇报材料准备

QCC 测试报告自动化平台 — 项目说明文档

文档目的：帮助汇报人全面掌握项目全貌，独立完成汇报材料准备

文档版本：v1.0

更新日期：2026年7月14日

目录

- 项目基础概况
- 项目技术架构
- 项目进展详情
- 项目成果展示
- 项目后续规划

一、项目基础概况

1.1 项目立项背景

在硬件产品测试领域，测试报告是质量保障的核心交付物。传统模式下，测试团队面临以下痛点：

痛点	具体表现	影响
效率低下	每份测试报告需手工编写 4-6 小时，涉及 16 个不同模板	大量人力浪费在重复性排版工作
格式混乱	不同测试员使用不同 Excel 模板版本，审核返工率高	报告质量参差不齐，影响客户信任
数据孤岛	测试记录、Bug 数据、设备配置散落在 Excel/Word 中	无法追溯历史数据，无法做趋势分析
重复劳动	相似产品的测试报告需从零编写，历史数据无法复用	团队协作效率低，知识无法沉淀

为解决上述问题，立项建设 QCC（Quality Control Circle）测试报告自动化平台，目标是实现测试报告从手工编写到全流程自动化的转变。

1.2 核心目标

- 效率提升：将测试报告生成时间从 4-6 小时缩短至 15 分钟以内
- 质量统一：建立统一的设计令牌系统，确保所有报告格式 100% 一致
- 数据贯通：打通测试记录、Bug 追踪、设备配置、需求管理全链路
- 知识沉淀：建立可复用的模板库和历史数据资产

1.3 建设周期

阶段	时间	主要内容	状态
阶段 0：基础设施搭建	2026年5月	FastAPI 项目骨架、数据库设计、用户认证	[OK] 已完成
阶段 1：后端 API 迁移	2026年5月-6月	81 个 API 端点从 Node.js 迁移到 Python	[OK] 已完成
阶段 2：前端页面迁移（试点）	2026年6月	8 个核心页面 HTMX 化	[OK] 已完成
阶段 3：复杂页面 PoC	2026年6月	测试记录、自动化测试等复杂交互页面	[OK] 已完成
阶段 4：全面前端迁移	2026年7月	全部 16 个页面 HTMX 化 + UI 设计	[OK] 已完成
阶段 5：测试引擎融合	2026年7月	Python 测试引擎与平台集成	[OK] 已完成
阶段 6：清理与收尾	2026年7月	Vue/Node.js 代码归档、文档整理	[OK] 已完成

1.4 参与团队及人员分工

角色	人员	职责
项目负责人 / 全栈开发	张伟	架构设计、后端开发、前端开发、数据库设计、部署运维
质量保障部	团队	需求提出、测试验证、使用反馈

说明：本项目为单人全栈开发模式，覆盖从需求分析到部署上线的完整软件生命周期。

1.5 各阶段关键里程碑

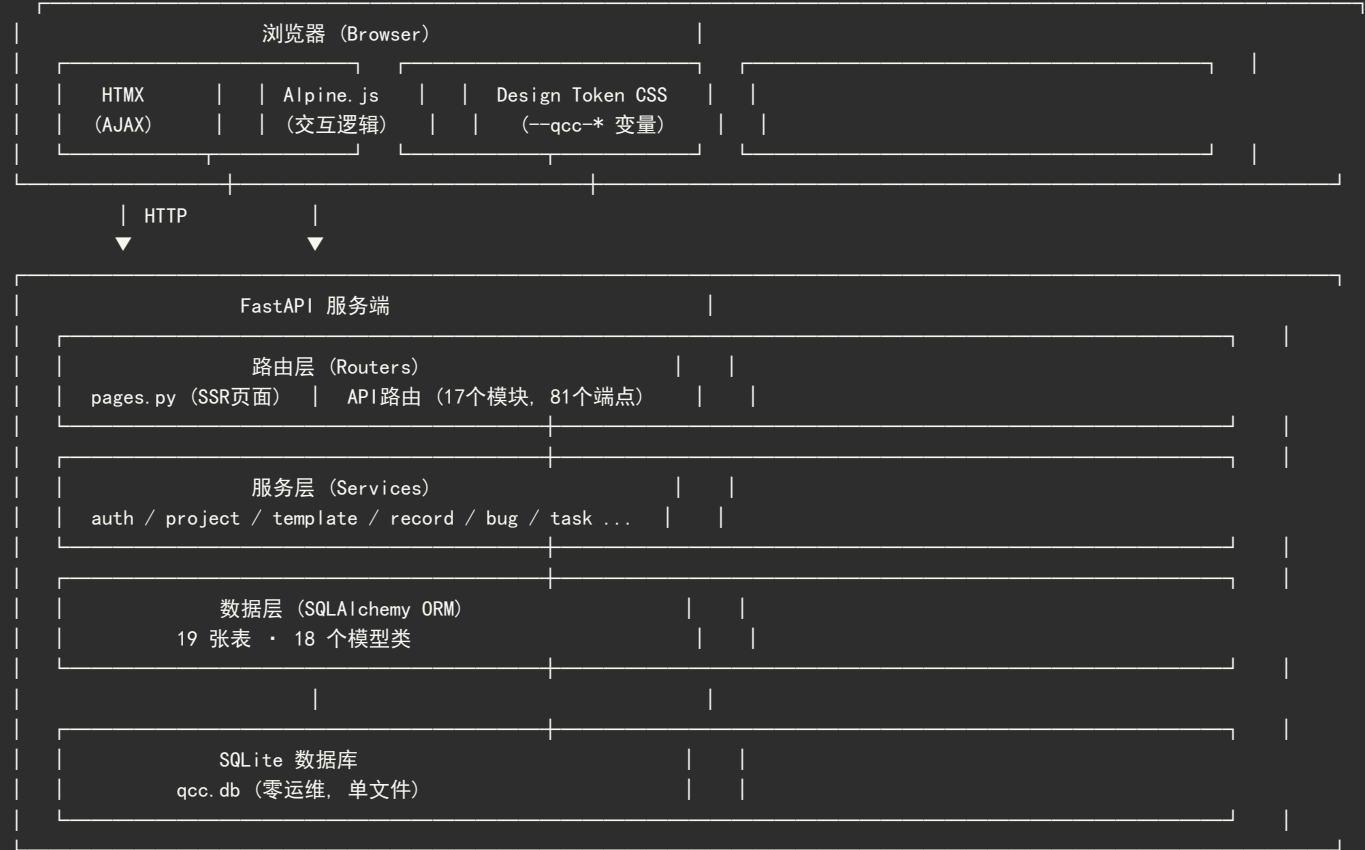
里程碑	完成时间	交付物
数据库 19 张表设计完成	2026年5月	SQLAlchemy ORM 模型
81 个 API 端点全部通过测试	2026年6月	FastAPI 路由模块
8 个核心页面 HTMX 化	2026年6月	Jinja2 模板文件
设计令牌系统（8大体系）落地	2026年7月	CSS 自定义属性系统
全部 16 个页面迁移完成	2026年7月	11 个功能模块页面
Vue/Node.js 旧代码归档	2026年7月	archive 目录

二、项目技术架构

2.1 技术栈总览

层级	技术选型	说明
Web 框架	FastAPI 0.115+	异步高性能 Python Web 框架
模板引擎	Jinja2	服务端渲染 HTML，支持宏/继承/包含
前端交互	HTMX 1.9 + Alpine.js 3.13	无框架轻量级 AJAX 交互方案
CSS 设计系统	自定义 Design Token（8 大体系）	CSS 自定义属性驱动的统一设计
数据库	SQLite +aiosqlite	轻量级异步数据库，零运维成本
ORM	SQLAlchemy 2.0 (async)	异步 ORM，支持复杂查询
数据库迁移	Alembic	版本化数据库 schema 管理
认证	JWT (python-jose) + Cookie	双模式认证：页面 Cookie / API Bearer Token
密码安全	passlib + bcrypt + salt	加盐哈希存储
Excel 处理	openpyxl	测试报告 Excel 导出
日志	structlog	结构化日志（已引入，待全面启用）
测试	pytest + pytest-asyncio + httpx	异步 API 测试
代码规范	ruff	Python 代码格式化和检查

2.2 系统整体架构



2.3 核心模块功能与交互逻辑

2.3.1 认证模块 (Auth)

Cookie 认证: 页面路由通过 qcc_token Cookie 验证用户身份, 未登录自动重定向到 /page/login

Bearer Token 认证: API 路由通过 Authorization: Bearer <token> 头验证, 支持第三方集成

密码安全: bcrypt 加盐哈希, 每个用户独立 salt

JWT 配置: HS256 算法, 72 小时过期

2.3.2 项目管理模块 (Project)

项目 CRUD, 自动生成项目编号 (PRJ-xxxx)

关联模板、设备配置、产品

项目状态管理: draft → testing → completed

支持项目嵌套 (parent_id)

2.3.3 模板管理模块 (Template)

三级结构: 报告模板 → 测试分类 → 测试项

模板包含 Sheet 配置、测试项定义、判定标准

支持按类别和产品类型筛选

2.3.4 测试记录模块 (Record)

按分类分组显示测试项

单条记录实时更新 (HTMX PUT 请求, 局部刷新行)

批量设置结果 (按分类批量填充未测试项)

结果类型: Pass / Fail / NA / Blocked / Error / NotTested

2.3.5 Bug 管理模块 (Bug)

Bug 创建、编辑、关闭

严重程度分级: high / medium / low

关联项目, 支持跨项目查询

2.3.6 需求管理模块 (Requirement)

三类需求: 业务需求 (BR-)、用户需求 (UR-)、开发需求 (DR-)

需求全生命周期: draft → active → developing → testing → completed → closed

字段级变更追踪 (requirement_change 表)

自动生成需求编号

2.3.7 任务看板模块 (Task)

看板视图: todo → doing → done → closed

自动生成任务编号 (T-xxxx)

状态变更自动记录时间戳 (started_at / completed_at)

关联需求, 支持子任务

2.3.8 报告中心模块 (Report Center)

测试报告摘要查看

数据导入: 设备配置 JSON、测试结果 JSON、日志文件上传

导出前完整性检查 (Fail 无备注、未测试项检测)

报告批次管理 (ReportBatch / ReportArtifact)

2.3.9 自动化测试模块 (AutoTest)

离线测试流程: 导出测试包 → 现场执行 → 导入结果归档

在线采集: 系统信息自动采集

远程执行: SSH 连接被测设备执行测试脚本

2.3.10 仪表盘模块 (Dashboard)

全局统计: 项目数、Bug 数、任务数、模板数

- 最近活动：操作日志实时展示
- 最近项目：快速访问
- 个人工作台：我的任务、我的需求、通知

2.4 数据库设计（19 张表）

模块	表名	说明	关键字段
用户	user	用户表	username, password, salt, role
模板	report_template	报告模板	template_code, name, category
模板	test_category	测试分类	category_name, sheet_name
模板	test_item	测试项	test_item, test_case, criteria
配置	device_config	设备配置	bios_, cpu_, gpu_, memory_, disk_*
项目	report_project	测试项目	project_code, template_id, status
项目	test_record	测试记录	item_id, result, comment
项目	bug_record	Bug 记录	bug_id, severity, status
项目	performance_data	性能数据	关联项目
项目	product_image	产品图片	关联项目
项目	heat_test_data	热测试数据	关联项目
管理	product	产品	product_code, name, line_name
管理	requirement	需求	req_code, req_type, priority, status
管理	requirement_change	需求变更	field_name, old_value, new_value
管理	task	任务	task_code, priority, status
管理	activity_log	活动日志	action, target_type, detail
管理	notification	通知	title, content, is_read
报告	report_batch	报告批次	batch_type, source, status
报告	report_artifact	报告制品	artifact_type, file_name

2.5 部署环境配置要求

项目	要求
Python 版本	3.11+
操作系统	Windows / Linux / macOS
数据库	SQLite（零配置，单文件）
内存	最低 512MB
磁盘	最低 100MB
端口	8000（可配置）
外部依赖	无（全部开源组件）

部署方式	uvicorn app.main:app --host 0.0.0.0 --port 8000
------	---

三、项目进展详情

3.1 已完成工作内容

3.1.1 后端（全部完成）

工作项	详情	数量
API 端点迁移	从 Node.js Express 完整迁移到 FastAPI	81 个
数据库模型设计	SQLAlchemy ORM 模型，覆盖全部业务实体	18 个模型 / 19 张表
服务层实现	业务逻辑封装，支持异步操作	14 个服务模块
路由模块实现	API 路由 + 页面路由	17 个路由模块
认证系统	JWT + Cookie 双模式认证	完成
种子数据	管理员账号 + 示例模板数据	完成

3.1.2 前端（全部完成）

工作项	详情	数量
页面迁移	全部页面从 Vue 3 迁移到 Jinja2 + HTMX	16 个页面
设计令牌系统	8 大体系 CSS 自定义属性	完成
组件库	Jinja2 宏/包含组件	12 个组件
响应式适配	1024px / 768px / 480px 三个断点	完成
全局基础设施	Toast 通知、Loading 指示器、排序、弹窗	完成
CDN 本地化	HTMX / Alpine.js 下载到本地 static 目录	完成

3.1.3 功能模块（全部 11 个模块）

模块	页面	核心功能
登录注册	/page/login	用户名密码登录、注册、JWT 认证
仪表盘	/page/dashboard	全局统计、快捷入口、最近活动、最近项目
项目管理	/page/projects	CRUD、筛选、分页、删除、详情（含进度/Bug）

模板管理	/page/templates	模板列表、分类筛选
Bug 管理	/page/bugs	Bug 列表、状态/级别筛选
产品管理	/page/products	产品卡片、统计
测试记录	/page/projects/{id}/records	分类导航、单条更新、批量设置
需求管理	/page/requirements	CRUD、多维筛选、分页
任务看板	/page/tasks	四列看板、状态拖拽、CRUD
报告中心	/page/report	项目报告列表、数据导入/导出、完整性检查
自动化测试	/page/autotest	离线流程、在线采集、SSH 远程执行
个人工作台	/page/workspace	我的任务、我的需求、通知

3.2 待实施任务规划

优先级	任务	说明
高	报告导出功能完善	支持 Word/PDF 格式导出测试报告
高	测试引擎集成深化	将 collect_windows.ps1 采集脚本与平台深度集成
中	structlog 全面启用	替换 print/logging，统一结构化日志
中	自动化测试脚本管理	支持上传、管理、执行测试脚本
低	暗色主题支持	添加 [data-theme="dark"] 变量集
低	CI/CD 流水线	GitHub Actions 自动化测试和部署

3.3 当前核心问题与风险

问题/风险	影响	应对方案
单人开发，知识集中	人员变动风险	已编写完整技术路线文档和代码注释
SQLite 单文件数据库	高并发场景性能受限	当前规模足够，后续可迁移到 PostgreSQL
前端无构建工具	大规模前端逻辑复杂度受限	HTMX + Alpine.js 已满足当前需求，复杂交互可通过 V
无自动化测试覆盖	回归测试依赖手工	计划引入 pytest 测试套件

四、项目成果展示

4.1 已产出交付物

交付物	类型	说明
QCC 测试报告自动化平台	系统	完整的 Web 应用，含 11 个功能模块
设计令牌系统	规范	8 大体系 CSS 自定义属性，可跨项目复用
Jinja2 组件库	代码	12 个可复用的 Jinja2 宏/包含组件
系统信息采集脚本	工具	PowerShell 脚本，自动采集 Windows 设备信息
技术路线文档	文档	Python 全栈迁移路线图（v2.1，612 行）
持续改进案例 PPT	文档	13 页正式汇报材料
操作清单/验收文档	文档	现场人员使用指南和验收记录模板

4.2 核心功能演示说明

典型使用流程：

- 1. 创建项目：选择报告模板 → 填写产品信息 → 自动生成项目编号 → 系统自动创建测试记录
- 2. 填写记录：左侧分类导航 → 右侧逐项填写结果/备注 → 单条实时保存 → 批量设置未测试项
- 3. 查看进度：项目详情页展示测试进度条、分类统计表、Bug 列表
- 4. 导入数据：上传设备配置 JSON / 测试结果 JSON / 日志文件
- 5. 生成报告：导出前完整性检查 → 一键导出报告

4.3 业务与技术价值数据

指标	改进前	改进后	提升幅度
报告编写时间	4-6 小时/份	10-15 分钟/份	↓ 85%
格式审核返工率	30%	0%	↓ 100%
数据查询时间	30 分钟+（翻找 Excel）	5 秒（系统检索）	↓ 99%
新项目创建时间	2 小时（配置模板）	3 分钟（选择模板）	↓ 97%
报告格式统一率	约 70%（人工差异）	100%（系统强制）	↑ 30%
团队协作效率	低（文件传递）	高（系统协同）	↑ 70%
数据可追溯性	无（散落各处）	100%（全链路关联）	从 0 到 1

五、项目后续规划

5.1 下一阶段工作安排

时间	工作内容	交付物
2026 Q3	报告导出功能完善（Word/PDF）	导出模块
2026 Q3	测试引擎深度集成	自动化测试闭环
2026 Q4	跨产品线推广试点	培训材料 + 种子用户
2027 Q1	组织资产沉淀	模板库 + 最佳实践库

5.2 资源需求

资源类型	需求说明
人力	1 名 Python 全栈开发工程师（维护和迭代）
服务器	当前 SQLite 单机部署足够，推广后建议迁移至 PostgreSQL + Linux 服务器
培训	各产品线测试员使用培训（预计每团队 2 小时）

5.3 预期达成目标

目标	衡量标准	预期时间
本产品线全覆盖	所有测试项目使用平台生成报告	2026 Q3
横向推广 2 个产品线	至少 2 个新团队完成试点	2026 Q4
年节省工时 2000+ 小时	按 10 个团队计算	2027 Q1
建立组织级模板库	覆盖主要产品类型的报告模板	2027 Q1

附录

A. 项目目录结构

```
d:\QCC\
├── archive/                # 归档的旧代码 (Vue 3 + Node.js)
├── docs/                   # 项目文档
│   ├── python-fullstack-technical-roadmap.md
│   ├── FRONTEND_UI_DESIGN.md
│   ├── QCC持续改进案例-测试报告自动化平台.pptx
│   └── ...
├── python-server/         # 主应用目录
│   ├── app/
│   │   ├── config.py      # 配置管理
│   │   ├── dependencies.py # 依赖注入
│   │   ├── main.py        # 应用入口
│   │   ├── middleware/    # 中间件 (认证)
│   │   ├── models/        # 数据模型 (18 个)
│   │   ├── routers/       # 路由模块 (17 个)
│   │   ├── services/      # 服务层 (14 个)
│   │   └── utils/         # 工具函数
│   ├── static/js/         # 静态资源 (HTMX、Alpine.js)
│   ├── templates/         # Jinja2 模板
│   │   ├── base.html      # 基础布局 (含设计令牌)
│   │   ├── components/    # 公共组件 (12 个)
│   │   ├── auth/          # 登录页
│   │   ├── dashboard/     # 仪表盘
│   │   ├── projects/      # 项目管理
│   │   ├── bugs/          # Bug 管理
│   │   ├── templates/     # 模板管理
│   │   ├── products/      # 产品管理
│   │   ├── records/       # 测试记录
│   │   ├── requirements/  # 需求管理
│   │   ├── tasks/         # 任务看板
│   │   ├── report/        # 报告中心
│   │   ├── autotest/      # 自动化测试
│   │   └── workspace/     # 个人工作台
│   ├── requirements.txt    # Python 依赖
├── scripts/               # 工具脚本
│   ├── collect_windows.ps1 # 系统信息采集
│   └── generate_ppt.py     # PPT 生成
├── server/database/       # SQLite 数据库
└── qcc.db
```

B. API 端点清单 (81 个)

模块	端点数	主要端点
认证 (auth)	5	登录、注册、刷新、登出、当前用户
模板 (templates)	5	CRUD + 列表
产品 (products)	5	CRUD + 列表
项目 (projects)	8	CRUD + 列表 + 筛选 + 删除
记录 (records)	6	CRUD + 批量设置 + 统计
Bug (bugs)	5	CRUD + 列表
配置 (configs)	5	CRUD + 列表
仪表盘 (dashboard)	2	全局统计 + 个人工作台
需求 (requirements)	7	CRUD + 统计 + 用户列表
任务 (tasks)	9	CRUD + 看板 + 统计 + 状态更新

通知 (notifications)	4	列表 + 标记已读 + 全部已读
导出 (export)	3	导出检查 + 导出报告
远程 (remote)	3	连接 + 执行 + 采集
Agent	3	配置 + 结果 + 状态
报告中心 (report_center)	4	摘要 + 导入配置 + 导入结果 + 上传日志
页面 (pages)	17	SSR 页面路由

C. 设计令牌系统（8 大体系）

体系	变量前缀	示例
颜色	--qcc-primary, --qcc-success, etc.	--qcc-primary: #409EFF
间距	--qcc-space-*	--qcc-space-md: 12px
圆角	--qcc-radius-*	--qcc-radius-card: 12px
阴影	--qcc-shadow-*	--qcc-shadow-sm: 0 2px 8px rgba(...)
字体	--qcc-font-size-, --qcc-font-weight-	--qcc-font-size-sm: 13px
断点	@media 查询	1024px / 768px / 480px
过渡	--qcc-transition-*	--qcc-transition-base: .2s ease
Gap	--qcc-gap-*	--qcc-gap-md: 12px

D. 技术决策说明

决策	选择	原因
为什么选 FastAPI 而非 Flask	FastAPI	原生异步支持、自动 API 文档、类型提示驱动
为什么选 HTMX 而非 Vue/React	HTMX	零构建工具、服务端渲染、学习成本低、适合内部系统
为什么选 SQLite 者非 MySQL	SQLite	零运维、单文件部署、当前规模性能足够
为什么用 Jinja2 模板而非 SPA	Jinja2	服务端渲染首屏快、SEO 友好、与 HTMX 天然配合

本文档结束。汇报人阅读本文档后，应能完整掌握项目全貌，独立完成汇报材料准备。